Low-dimensional IPA hybrid certificate example

Example `controlled_d3_n2_seed2_a0.35_s0.45_c0.25`. Values are from the generated profiling artifact.

1 Starting matrices and problem formulation

The matrix family is

$$\mathcal{A} = \{A_1, A_2\} \subset \mathbb{R}^{3\times 3}.$$

For this example the invariant-polytope algorithm is initialized with the spectrum-maximizing product

$$\Pi = A_1, \qquad \rho(\Pi) = 1, \qquad \lambda = 1.$$

The computational problem is to certify

$$\mathrm{JSR}(\mathcal{A}) = 1.$$

The matrices used by the run are

$$A_1 = \begin{pmatrix} 0.28608433 & 0.03859008 & 0.03387308 \\ 0.03859008 & 0.99003830 & -0.07403809 \\ 0.03387308 & -0.07403809 & 0.33637737 \end{pmatrix},$$

$$A_2 = \begin{pmatrix} 0.16701638 & -0.00114923 & 0.02347436 \\ 0.05850404 & 0.20641451 & 0.02404550 \\ 0.01739058 & 0.03269350 & 0.18344411 \end{pmatrix}.$$

Since A_1 has spectral radius one, the hybrid certificate uses A_1 as the generator. The stored infinite-path leaf word is

$$(-1, 2).$$

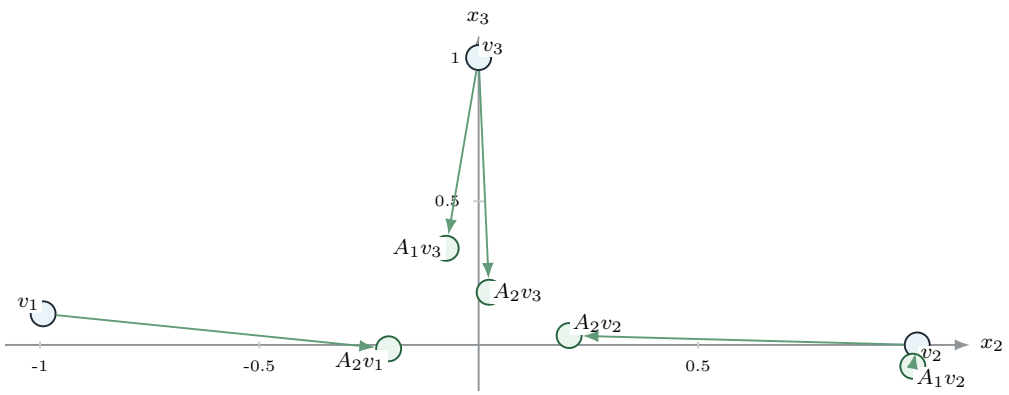
Here the negative index -1 denotes the generator direction associated with A_1, and the positive index 2 denotes the original matrix A_2.

2 Used starting vertices

The first vertex is the leading eigenvector of A_1. Two extra vertices are added explicitly.

$$v_1 = \begin{pmatrix} -0.04853357 \\ -0.99293274 \\ 0.10830080 \end{pmatrix}, \qquad v_2 = e_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \qquad v_3 = e_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

The following diagram shows the initial vertices and their one-step images under A_1 and A_2. The picture uses the (x_2, x_3)-projection, so it is a diagnostic plot rather than a geometric proof.



3 How the certificate is evaluated for three IPA versions

The baseline run uses ordinary IPA behavior. It generates vertices and evaluates exact polytope norms. The two hybrid runs construct the same initial cyclic trees but then try to close frontier vertices by the infinite-path certificate. The summary below is measured from the same profiling run.

Mode	vertices	generated	exact calls	exact points	hybrid closed	hybrid checks
baseline	7	13	2	4	0	0
strict	3	5	0	0	5	5
performance	3	5	0	0	2	2

For a candidate frontier vertex x, the hybrid code evaluates the finite part, the limiting branch, and a tail bound. The recorded result has the form

closed if `max_upper_norm`$(x) < 1$ and the stored tail bound is below the admissible threshold.

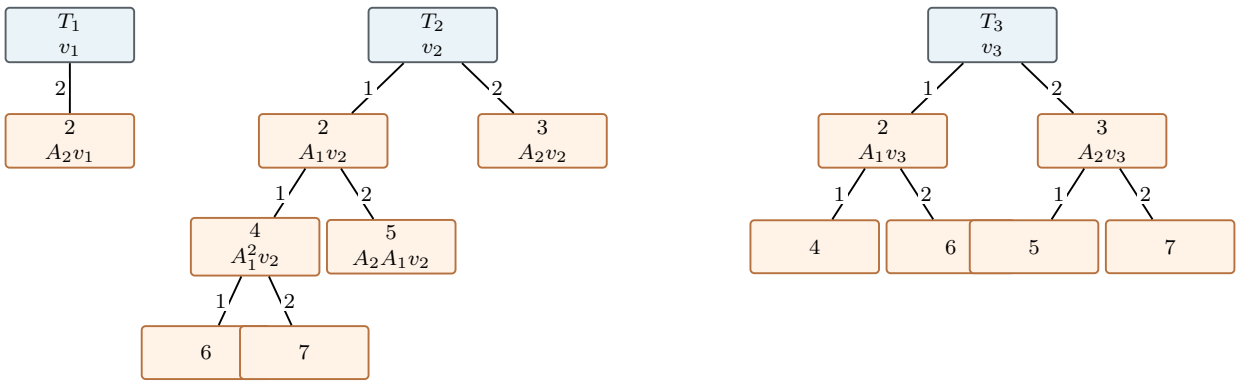
In this example every hybrid check closes. Strict mode evaluates every frontier vertex. Performance mode batches the checks and stops after the cheaper checks already close the necessary frontier vertices.

Mode	tree	vertex	max upper norm	tail N	tail bound
strict	1	2	0.04640145	0	0.37997383
strict	2	2	0.22752354	0	0.37017767
strict	2	3	0.05685588	1	0.20662863
strict	3	2	0.65556671	2	0.42480253
strict	3	3	0.37907819	2	0.24092434
performance	2	2	0	0	0.37017767
performance	3	2	0	2	0.42480253

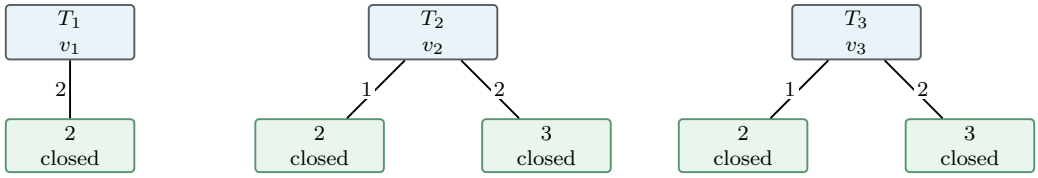
4 Used trees

The three cyclic trees are rooted at v_1, v_2, v_3. The edge labels are the indices of A_1 and A_2. The baseline has to grow trees T_2 and T_3 beyond depth one. The hybrid runs keep only the shallow frontier and certify it through the leaf word $(-1, 2)$.

Baseline trees



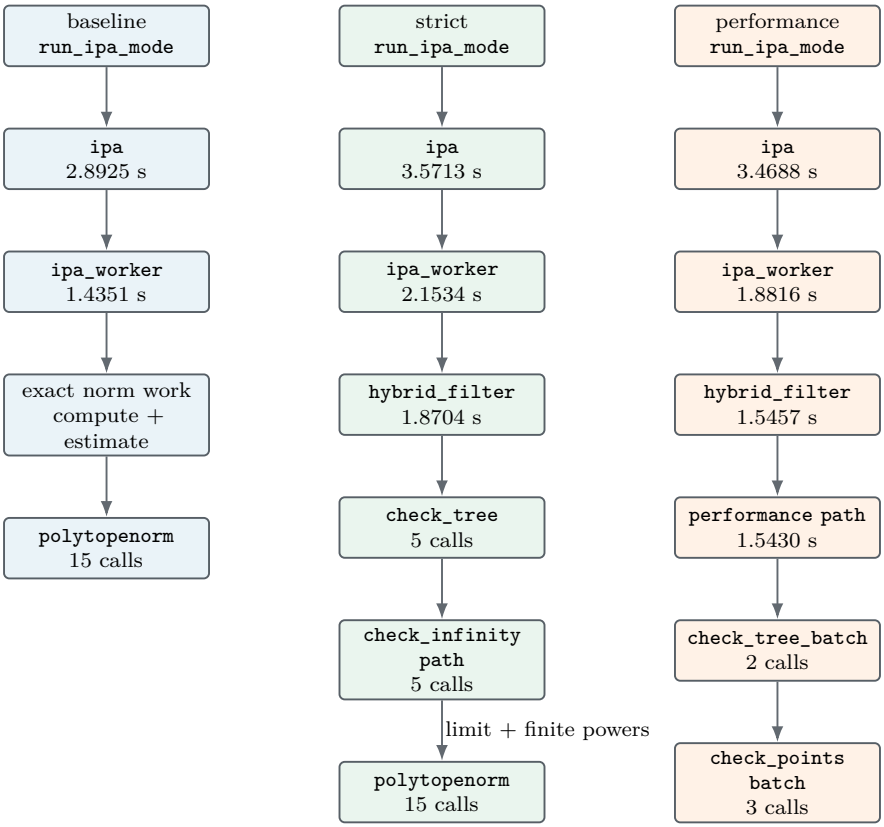
Hybrid trees



Strict checks all five green leaves. Performance closes leaves $T_2 : 2$ and $T_3 : 2$ after batching.

5 Call flow

The next diagram uses the measured MATLAB profiler call graph. Times are wall-clock times attributed to the corresponding call edge in the low-dimensional run. The baseline spends time in exact norm work. The hybrid runs enter `ipa_hybrid_filter_vertices`; strict checks each tree separately, while performance uses a batched path.



Mode	wall time	profiler functions	line hits	exact norm points
baseline	2.89299 s	473	120244	4
strict	3.57157 s	517	171573	0
performance	3.46920 s	525	119110	0